Serverless REST API Project using AWS

Manara Project

1. Project Overview

This project demonstrates the design and implementation of a fully serverless REST API using Amazon Web Services (AWS). The system allows users to perform CRUD (Create, Read, Update, Delete) operations on a task management system.

The architecture is designed to be scalable, cost-efficient, and highly available without managing any servers.

---

2. Architecture Description

The system follows serverless, event-driven architecture. Users access the application through a web interface hosted on Amazon S3, which is distributed globally using Amazon CloudFront.

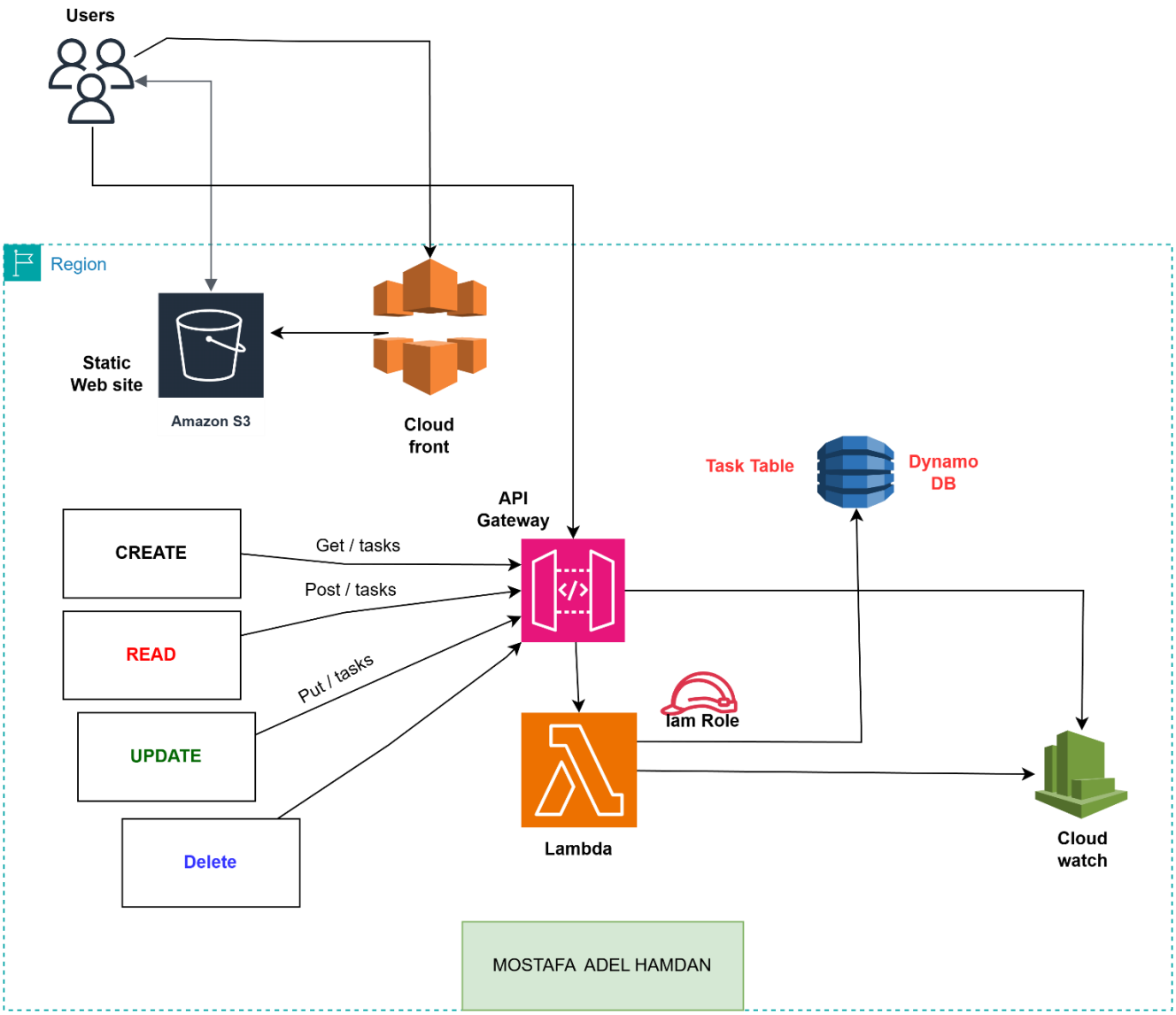
All API requests are routed through Amazon API Gateway, which triggers AWS Lambda functions to process the requests and interact with Amazon DynamoDB for data storage.

Monitoring and logging are handled by Amazon CloudWatch, while access control is managed through AWS Identity and Access Management (IAM).

---

3. Architecture Design and Flow

1. The user accesses the web application via Amazon CloudFront.
2. CloudFront delivers static content from Amazon S3.
3. The frontend sends HTTP requests (GET, POST, PUT, DELETE) to Amazon API Gateway.
4. API Gateway invokes AWS Lambda functions.
5. Lambda processes the request and interacts with DynamoDB.
6. Data is stored/retrieved from DynamoDB.
7. Logs and metrics are captured in CloudWatch.



---

4. AWS Services Used

4.1 Amazon S3

Amazon S3 is an object storage service that offers industry-leading scalability, data availability, security, and performance. It allows users to store and retrieve any amount of data at any time. In this project, S3 is used to host static website content such as HTML, CSS, and JavaScript files. It supports static website hosting and integrates seamlessly with other AWS services.

4.2 Amazon CloudFront

Amazon CloudFront is a global content delivery network (CDN) that securely delivers data, videos, applications, and APIs to users with low latency. It caches content at edge locations worldwide to improve performance.

In this architecture, CloudFront distributes the frontend hosted on S3 and enhances security by limiting direct access to the S3 bucket.

4.3 Amazon API Gateway

Amazon API Gateway is a fully managed service that enables developers to create, publish, maintain, monitor, and secure APIs at any scale. It acts as the entry point for client requests and supports RESTful APIs.

In this project, it handles HTTP methods such as GET, POST, PUT, and DELETE and routes requests to AWS Lambda functions.

Functions:

- Handles HTTP requests (CRUD operations)
- Routes requests to Lambda
- Supports request validation and throttling

Endpoints Example:

- GET /tasks
- POST /tasks
- PUT /tasks
- DELETE /tasks

4.4 AWS Lambda

AWS Lambda is a serverless compute service that runs code in response to events without provisioning or managing servers. It automatically scales based on incoming requests and charges only for execution time. In this project, Lambda functions process API requests and implements business logic for CRUD operations while interacting with DynamoDB.

Responsibilities:

- Process API requests
- Perform CRUD operations
- Communicate with DynamoDB

Advantages:

- Fully managed
- Auto-scaling
- Pay-per-use

4.5 Amazon DynamoDB

Amazon DynamoDB is a fully managed NoSQL database service that provides fast and predictable performance with seamless scalability. It stores data in key-value and document formats. In this system, DynamoDB is used to store task records, allowing efficient read and write operations with low latency.

Design:

- Table: Tasks
- Primary Key: task Id

Features:

- High performance
- Fully managed
- Automatic scaling

4.6 AWS Identity and Access Management (IAM)

IAM is used to control access between AWS services.

Usage in this project:

- Lambda execution role
- Permissions to access DynamoDB
- Secure interaction between services

4.7 Amazon CloudWatch

Amazon CloudWatch is a monitoring and observability service that provides data and insights for AWS resources and applications.

It collects logs, metrics, and events in real time. In this architecture, CloudWatch is used to monitor API performance and log Lambda executions for debugging and analysis.

---

5. Key Features of the System

- Fully serverless architecture
- Scalable and highly available
- Secure access using IAM roles
- Real-time logging and monitoring
- Global content delivery using CloudFront

---

6. Learning Outcomes

- Understanding serverless architecture design
- Building REST APIs using API Gateway and Lambda
- Working with NoSQL databases (DynamoDB)
- Implementing secure access using IAM
- Monitoring applications using CloudWatch

---

7. Conclusion

This project demonstrates a modern cloud-native approach to building scalable applications using AWS serverless services. It eliminates the need for infrastructure management while ensuring high performance, security, and reliability.

The architecture is suitable for real-world applications and can be extended with additional features such as authentication, caching, and CI/CD pipelines.